



OOP Final Term Project

BS Data Science — Semester 2

**Library Analyzed
requests**

Object-Oriented Design Analysis

Group Members	
Ameer Hamza	F25BDATS1M02077
Humayon Zahid	F25BDATS1M02064
Fahad Iqbal	S25BDATS1E01001
Rehman Ali Chattha	F25BDATS1M02068

Submitted to: Dr. Akmal Khan

Date: 28 May 2026



1. Library Overview

• What is the requests Library?

The requests library is a Python library that helps your program talk to websites and APIs on the internet. Whenever you want to get data from a website, send information, or use an API, you use requests. It is called 'HTTP for Humans' because its syntax is simple and beginner-friendly. Users can work with APIs easily without understanding low-level networking details.

• Who Uses It?

- ▶ Data Scientists — to download data from APIs for analysis
- ▶ Web Developers — to call APIs inside their programs
- ▶ Automation Engineers — for web scraping and automated testing
- ▶ DevOps Engineers — to talk to cloud services like AWS and Azure

• How to Install

```
pip install requests
```

• Basic Example

```
import requests

# Get data from GitHub API
response = requests.get('https://api.github.com')

print(response.status_code)    # 200 means success
print(response.json())         # shows the data as JSON
```

The library simplifies HTTP communication by handling connections, responses, and errors internally. The library handles all the hard work like making the connection, dealing with errors, and reading the response. This simplicity is one of the main reasons for its popularity among Python developers.

2. Class Hierarchy Diagram

The requests library uses OOP to organize its code into classes. Below is a table showing all main classes, their files, and inheritance relationships.

• Class Overview Table

Class Name	File	Type	Inherits From	Main Job
object	Python built-in	Root	—	Root of all Python classes
AuthBase	auth.py	Abstract	object	Base for all auth types
SessionRedirectMixin	sessions.py	Mixin	object	Handles redirects
BaseAdapter	adapters.py	Abstract	object	Defines the send() interface
HTTPBasicAuth	auth.py	Concrete	AuthBase	Username / password auth
Session	sessions.py	Concrete	SessionRedirectMixin	Sends HTTP requests
HTTPAdapter	adapters.py	Concrete	BaseAdapter	Makes real HTTP connections
LoggedSession ★	working.py	Custom	Session	Our own extension class

★ *LoggedSession is our custom extension class – see Section 6 for full code.*

• Inheritance Chains

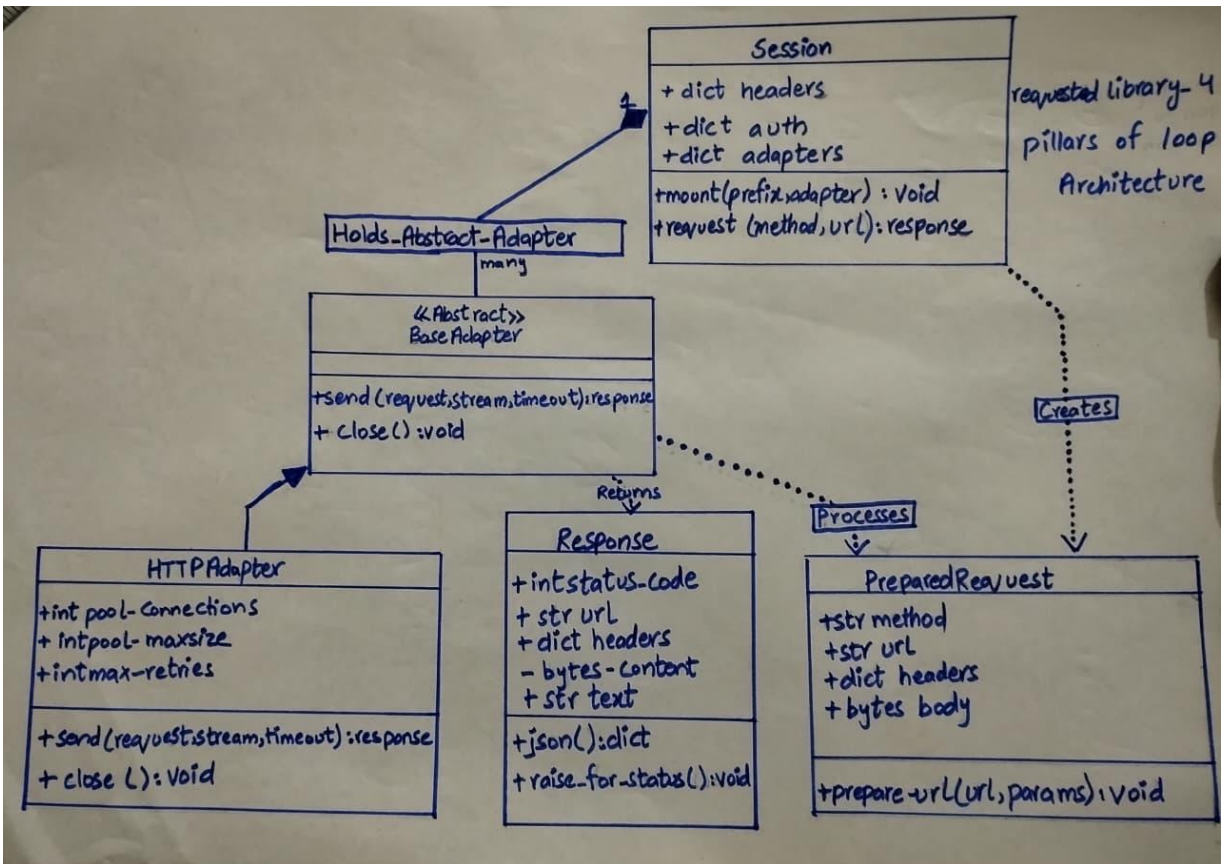
- ▶ Auth Chain: AuthBase → HTTPBasicAuth
- ▶ Adapter Chain: BaseAdapter → HTTPAdapter
- ▶ Session Chain: SessionRedirectMixin → Session → LoggedSession

• How Classes Use Each Other

- ▶ Session uses HTTPAdapter to send requests (mounted with session.mount())
- ▶ Session uses HTTPBasicAuth for login credentials (set via session.auth)

• Hand-Drawn UML Diagram

UML Class Diagram — requests Library



3. OOP Principles Analysis

The requests library is a great real-world example of all four OOP principles. Each principle is explained below with simple words and a real code example from the library source code.



Principle 1 — Encapsulation

Encapsulation means keeping data safe inside a class. Internal data is controlled through class methods to protect the object's state. The class controls everything through its own methods.

► Where it is in requests — Session class (sessions.py)

The Session class keeps all its data like cookies, headers, and adapters inside itself. The user does not manage these directly. The class handles them internally.

```
class Session:
    def __init__(self):
        self.headers = {}      # public — user can read and set
        self.cookies = {}     # public — but managed inside class
        self.auth = None      # public — user sets it once
        self.adapters = {}    # internal — only changed via mount()
```

The user never touches the HTTP connection directly. All complex work happens inside the class. This is Encapsulation — The internal implementation details are hidden from the user while only necessary functionality is exposed.



Principle 2 — Inheritance

Inheritance means a child class gets all the properties and methods of a parent class automatically. Inheritance improves code reusability by allowing child classes to reuse parent class functionality.

► Where it is in requests — Auth and Adapter classes

```
# Example 1 — Auth Inheritance (auth.py)
class AuthBase:                          # parent class
    def __call__(self, r):
        raise NotImplementedError        # child must write this

class HTTPBasicAuth(AuthBase):            # child class
    def __init__(self, username, password):
```

```

        self.username = username
        self.password = password
    def __call__(self, r):          # overrides parent
        r.headers['Authorization'] = ...
        return r

```

```

# Example 2 – Adapter Inheritance (adapters.py)
class BaseAdapter:                # parent class
    def send(self, request, **kwargs):
        raise NotImplementedError

class HTTPAdapter(BaseAdapter):    # child class
    def send(self, request, **kwargs):
        ... # real HTTP connection happens here

```

HTTPBasicAuth gets everything from AuthBase and just adds the login logic. HTTPAdapter gets everything from BaseAdapter and adds the real HTTP connection code. This is Inheritance.



Principle 3 — Polymorphism

Polymorphism means the same method interface can perform different behaviors depending on the class implementation. This makes code flexible and easy to extend.

► Where it is in requests — send() and __call__() methods

```

# send() – same name, different behaviour
class BaseAdapter:
    def send(self, request, **kwargs):
        raise NotImplementedError          # just a template

class HTTPAdapter(BaseAdapter):
    def send(self, request, **kwargs):
        # makes real TCP/SSL connection
        return self.build_response(request, resp)

```

```

# __call__() – same name, different auth behaviour
auth = HTTPBasicAuth('user', 'pass')
auth(request)    # adds Basic Authorization header

```

Our LoggedSession class also uses Polymorphism. We override the request() method – same method name as Session, but now it also writes a log entry every time a request is made.



Principle 4 — Abstraction

Abstraction means hiding all the complex work inside the class and only showing the user a simple interface. The implementation complexity is hidden behind a simple public interface.

► Where it is in requests — BaseAdapter as abstract interface

```
class BaseAdapter:
    """Base class for all Transport Adapters"""

    def send(self, request, **kwargs):
        raise NotImplementedError # child class must do the work

    def close(self):
        raise NotImplementedError
```

From the user's side, all they ever write is one line:

```
response = requests.get('https://example.com')
```

They do not see: creating a PreparedRequest, finding the right adapter, opening a TCP socket, handling SSL, following redirects, or reading the response. All of that complex work is abstracted away inside the library.

4. Design Decision Critique

• Decision We Chose: Session Class Does Too Much

We chose to critique the Session class because it has too many responsibilities inside one class. This goes against the Single Responsibility Principle (SRP) from the SOLID design rules in software engineering.

• What Session Currently Does

- Stores and manages cookies
- Stores and manages request headers
- Handles user authentication
- Handles HTTP redirects (using SessionRedirectMixin)
- Mounts and manages transport adapters
- Sends HTTP requests
- Manages connection pooling and keep-alive

This combines multiple responsibilities into a single class. The Single Responsibility Principle says: one class should have only one reason to change. Large multi-purpose classes are generally harder to maintain, debug, and test.

• Trade-offs

Aspect	Current Design (One Session)	Better Design (Separated)
Ease of Use	Very easy — one object does all	Slightly harder to set up
Maintainability	Hard — class is 600+ lines long	Easy — each class is small
Testability	Hard to test each feature alone	Each part can be tested separately
Flexibility	Low — parts are tightly connected	High — easy to swap or extend
Code Reuse	Limited	Individual parts can be reused

• Our Suggested Alternative

```
class CookieManager:    # only manages cookies
class AuthManager:     # only manages authentication
class RedirectHandler: # only handles redirects
class AdapterRegistry: # only manages adapters

class Session:         # uses all of them together
    def __init__(self):
        self.cookies = CookieManager()
        self.auth = AuthManager()
```

```
self.redirect = RedirectHandler()  
self.adapters = AdapterRegistry()
```

The public API would stay exactly the same for the user. But internally the code would be much cleaner, easier to maintain, and easier to test.

5. Comparison: requests vs httpx

We compared requests with httpx, a modern Python HTTP library designed to improve on requests. Both do the same basic job but are designed differently.

• Feature Comparison

Feature	requests	httpx
Main Class	Session	Client / AsyncClient
Async Support	No	Yes (async / await)
HTTP/2	No	Yes
Type Hints	Partial	Full
Ease of Use	Very Easy	Easy
Maturity	Very mature (2011)	Newer (2019)
Install	pip install requests	pip install httpx
Community	Very large	Growing fast

• OOP Design Differences

- ▶ requests uses Session as the main class with SessionRedirectMixin mixed in. httpx uses a separate Client for sync and AsyncClient for async – much cleaner design.
- ▶ httpx follows the Single Responsibility Principle better. Each class has one clear job. requests puts everything in one Session class.
- ▶ Both libraries use Abstraction — the user just calls get() or post() and does not see the HTTP internals.
- ▶ requests is easier to learn for beginners. httpx is better for modern production systems that need async or HTTP/2.

• Conclusion

For this project, requests was the right choice because it has a clear and well-known OOP structure that is easy to analyze and explain. httpx may be a better choice for modern systems that require asynchronous programming or HTTP/2 support.

6. Custom Extension — LoggedSession

• What We Built

We implemented a custom class named `LoggedSession` that extends the `Session` class from `requests`. It automatically saves a log of every HTTP request made. This is in a file called `working.py` and demonstrates the practical implementation of all four OOP principles.

• OOP Principles in Our Code

Principle	How We Used It
Inheritance	<code>LoggedSession</code> extends <code>Session</code> — gets <code>get()</code> , <code>post()</code> , <code>put()</code> and all other methods for free
Encapsulation	<code>_log</code> is private (starts with <code>_</code>). It cannot be changed from outside. Only <code>get_log()</code> can read it
Polymorphism	We override <code>request()</code> — same method name as <code>Session.request()</code> but now it also writes a log entry
Abstraction	User just calls <code>session.get()</code> or <code>session.post()</code> . They do not see the logging happening inside

• Full Code — `working.py`

```
import requests
from requests import Session
from requests.adapters import HTTPAdapter
from requests.auth import HTTPBasicAuth

class LoggedSession(Session):          # Inheritance
    """
    Custom extension of requests.Session.
    Logs every HTTP request automatically.
    Demonstrates all 4 OOP principles.
    """
    def __init__(self):
        super().__init__()             # call Session __init__
        self._log = []                 # Encapsulation — private

    def request(self, method, url, **kwargs):  # Polymorphism
        print(f'[LOG] {method.upper()} -> {url}')
        response = super().request(method, url, **kwargs)
        self._log.append({
            'method': method.upper(),
            'url': url,
            'status': response.status_code
        })
        return response
```

```

def get_log(self):                                # Getter – Encapsulation
    return self._log.copy()

def show_log(self):                                # Abstraction
    print('\n--- Request History ---')
    for i, e in enumerate(self._log, 1):
        print(f"{i}. {e['method']} {e['url']} -> Status {e['status']}")

def clear_log(self):
    self._log = []

if __name__ == '__main__':
    session = LoggedSession()
    adapter = HTTPAdapter(max_retries=3)
    session.mount('https://', adapter)
    session.auth = HTTPBasicAuth('testuser', 'testpass')
    session.get('https://httpbin.org/get')
    session.post('https://httpbin.org/post')
    session.get('https://httpbin.org/status/404')
    session.show_log()

```

• Output When You Run the Code

```

[LOG] GET -> https://httpbin.org/get
[LOG] POST -> https://httpbin.org/post
[LOG] GET -> https://httpbin.org/status/404

--- Request History ---
1. GET https://httpbin.org/get -> Status 200
2. POST https://httpbin.org/post -> Status 200
3. GET https://httpbin.org/status/404 -> Status 404

```

• Why This Extension is Useful

- ▶ Automatically tracks every API call without adding extra code in main program
- ▶ Very helpful for debugging — you can see which requests were made and their status
- ▶ The private `_log` cannot be accidentally changed from outside the class
- ▶ Works exactly like a normal Session — no changes needed in existing code

7. References

1. requests Official Documentation — <https://docs.python-requests.org>
2. requests Source Code on GitHub — <https://github.com/psf/requests>
3. httpx Official Documentation — <https://www.python-httpx.org>
4. Python OOP Tutorial (Official) — <https://docs.python.org/3/tutorial/classes.html>
5. Real Python — Using requests Library — <https://realpython.com/python-requests>
6. SOLID Principles in Python — <https://realpython.com/solid-principles-python>
7. AI tools were used only for concept clarification and formatting assistance.